Firelight

Audit Competition

Summary Report
December 2025

Content Index

- Overview ---------- 3
- Scope of Assets ---------- 4
- Summary ---------- 5
- Leaderboard ---------- 6
- Top 5 Reports ---------- 8

Overview

Immunefi Audit Competitions are special, time-limited events that supercharge the reach and visibility of programs to our whitehat community.

From November 7 to November 17, the Firelight Audit Competition offered up to $15,000 USD in rewards for security researchers.

Immunefi's Discord server hosted a channel for enhanced, two-way communication between whitehats and the Firelight team, improving feedback and response times. Managed Triaging was also activated for the duration of this event, streamlining the resolution process for incoming bug reports.

During this event, 6 low, and 40 insight reports were found on the target contracts. A total of 191 security researchers participated.

Firelight distributed the $15,000 USD reward pool to 46 of the very best submissions for the security researchers' valiant efforts, including "Insight" submissions scored on a rating system that takes into account levels of:

1) Security best practices
2) Code optimizations and enhancements
3) Architectural decentralization and composability
4) Documentation improvements

Firelight Introduction

The Firelight Vault is an upgradeable ERC-4626 compatible vault with additional features.

Scope Of Assets

The target assets in scope for the Audit Competition were Firelight's smart contracts.

The total nSLOC was 528.

Summary

Duration: **10 days**	Comp Date: **Nov 7 - Nov 17**	Rewards Pool: **$15,000 USD**	nSLOC: **528**
Submitted reports: **329**	Security researchers: **191**	Valid vulnerabilities: **6**	Insight reports: **40**

Total Whitehat Participation

191
Total Researchers

44
Paid Researchers

Leaderboard

Position	Reward	Username	Valids	Insights
1	$2,546	chief_hunter888	1	0
2	$1,640	EagleEye	1	0
3	$1,640	piyushmali	1	0
4	$1,516	Paludo0x	1	0
5	$1,516	perseverance	1	0
6	$1,145	blackgrease	1	0
7	$1,145	emilesean_es	1	0
8	$1,145	Flare0x	1	0
9	$1,145	gklptrgt	1	0
10	$213	Le_Rems	1	1
11	$213	Tadev	1	1
12	$197	ZenHunter	0	1
13	$197	hunterKing	0	1
14	$197	VinayVig	0	1
15	$73	dobrevaleri	1	0
16	$66	a16	0	1
17	$16	Tomioka	1	0
18	$16	Falendar	1	0
19	$16	dldLambda	1	0
20	$16	jayx	1	0
21	$16	Arkindyo	1	0
22	$16	coinsspor	1	0

Position	Reward	Username	Valids	Insights
23	16	theboiledcorn	1	0
24	16	axolot	1	0
25	16	edantes	1	0
26	16	legion	1	0
27	16	Diavol0	1	0
28	16	Y4nhu1	1	0
29	16	jesse03	1	0
30	16	zeroK	1	0
31	16	Orionn	1	0
32	16	maggie	1	0
33	16	sahuang	1	0
34	16	zcai	1	0
35	16	vivekd	1	0
36	16	y4y	1	0
37	16	sol_4th05	1	0
38	16	redbeans	1	0
39	16	Pro_King	1	0
40	16	hcrlen	1	0
41	16	Immanux2160	1	0
42	16	IronsideSec	1	0
43	0	[redacted]	1	0
44	0	[redacted]	1	0

Top 2 Reports

Implementation Contract Does Not Disable Initializers

Report number: 59091

Submitted by: @chief_hunter888

Target: https://github.com/firelight-protocol/firelight-core/blob/main/contracts/FirelightVault.sol

Impacts:

- Contract fails to deliver promised returns, but doesn't lose value

Program Action: Confirmed as low severity.

Report Excerpt:

The contract `FirelightVault.sol` is an `Initializable.sol` children, meant to be used behind a proxy. However, the current implementation has no constructor in which it would call the `_disableInitializers()` function as [recommended by the OpenZepplin, developer of the inherited contracts](https://docs.openzeppelin.com/upgrades-plugins/writing-upgradeable#initializing-the-implementation-contract). As a result anyone can just initialize the implementation contract and thereby take over the contract as the owner. Furthermore, due to the irreversible nature of initializable.sol, it can only be initialized once. This means once a malicious attacker initialized the contract, its storage/state is irrevocably corrupted.

Incorrect timestamp calculation in periodAtTimestamp() leads to broken historical period lookups

Report number: 58993

Submitted by: @dobrevaleri

Target: https://github.com/firelight-protocol/firelight-core/blob/main/contracts/FirelightVault.sol

Impacts:

- Contract fails to deliver promised returns, but doesn't lose value

Program Action: Confirmed as low severity.

Report Excerpt:

The `FirelightVault::periodAtTimestamp()` function incorrectly uses the current block timestamp instead of the provided `timestamp` parameter when calculating the period number. This causes the function to always return the current period regardless of which historical timestamp is queried, breaking any functionality that relies on historical period-to-timestamp mappings.

Disclaimer

This report is subject to the terms and conditions set forth in the Master Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report may be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement.

Customer may not transmit, disclose, refer to or rely upon this report for any purposes without Immunefi's prior written consent. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Immunefi to perform a security assessment. Immunefi does not provide any warranty or guarantee regarding the bug-free nature of the technology analyzed, nor does Immunefi provide any indication of approval of the technology's proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project.

Customer is solely responsible for its own due diligence and continuous security and compliance programs. Immunefi expressly disclaims any warranty associated with Services as summarized in this report. Customer acknowledges that this report does not represent an extensive security review and is intended solely to summarize the findings and statistics of the audit competition.

Immunefi

End of Report